

Guia completo para subir o BT-Barber em produção

Passo a passo do que clicar, onde clicar e o que você deve estar vendo em cada tela — Stripe, Google, Vercel, Supabase e domínio.

Versão: 1.0

Data: Maio de 2026

Para: Patrick (patrick.almeida@sttart.com.br)

Pré-requisito: conta Stripe já criada e KYC enviado

Índice

1	Ativar conta Stripe (modo Live)	~10 min — espera KYC 1-2 dias
2	Habilitar PIX e cartão	~5 min em Test, até 1 dia em Live
3	Configurar branding e descritor de extrato	~10 min
4	Pegar as API keys de produção	~2 min
5	Adicionar conta bancária para receber repasses	~5 min
6	Comprar/configurar domínio	~30 min (espera DNS propagar)
7	Subir código no GitHub	~5 min
8	Importar projeto na Vercel	~3 min
9	Configurar variáveis de ambiente na Vercel	~10 min
10	Conectar domínio na Vercel	~10 min
11	Criar webhook da Stripe (modo Live)	~5 min
12	Configurar redirect do Google OAuth para produção	~5 min
13	Preencher placeholders das páginas legais	~15 min
14	Verificar Vercel Cron	~2 min
15	Smoke test em produção	~10 min
16	Hardening pós-launch (recomendado)	~30 min

1 Dica: Cada passo lista o que clicar, o que você deve ver na tela e o que copiar / guardar pra usar nos passos seguintes. Os valores que precisam viajar entre passos (API keys, webhook secret, IDs) estão marcados em **negrito** — anote num bloco de notas seguro enquanto avança.

1 Ativar conta Stripe (modo Live)

Você já criou a conta. Agora precisa **ativar pra receber dinheiro de verdade** — em modo "Test" você só consegue simular pagamentos.

1.1 Entrar no Dashboard

1. Acessa <https://dashboard.stripe.com>
2. No canto superior direito, perto do seu nome, tem um toggle **TEST MODE**. Clica nele para virar **LIVE MODE** (cor laranja).

VOCÊ DEVE VER

O dashboard com uma faixa laranja no topo dizendo "Activate your account to start accepting live payments" e um botão azul "Activate account" ou "Start activation".

1.2 Preencher o formulário de ativação (KYC)

1. Clica em "Activate account". Vai abrir um wizard de 4-5 telas.
2. **Tipo de negócio:** escolhe entre Empresa (CNPJ) ou Profissional autônomo (CPF). Se ainda não tem CNPJ, pode começar como autônomo, mas a Stripe limita o volume — a partir de R\$ 5k/mês ela costuma exigir CNPJ.
3. **Razão social / Nome legal:** digite exatamente como está no contrato social ou no CPF. Esse texto aparece no extrato do PIX do cliente.
4. **CNPJ ou CPF:** o número.
5. **Endereço:** deve bater com o endereço fiscal cadastrado.
6. **Descrição do produto/serviço:** cola o texto abaixo (já ajustado pra Stripe Brasil — costuma ser aceito sem revisão extra):

Plataforma online (marketplace) que conecta clientes finais a barbearias parceiras no Brasil. Os clientes utilizam o site para:

(1) Agendar serviços de barbearia com profissionais cadastrados — corte de cabelo, barba, sobrancelha, hidratação e demais cuidados estéticos masculinos e femininos;

(2) Comprar produtos correlatos vendidos pelas barbearias parceiras — bebidas, cosméticos para cabelo e barba (pomadas, óleos, shampoos), e acessórios.

A plataforma processa o pagamento online (PIX e cartão de crédito) e repassa o valor às barbearias parceiras periodicamente, retendo uma taxa de

serviço sobre cada transação como contrapartida pela operação do marketplace.

7. **Site/URL do negócio:** o domínio que você vai usar (ex.: `https://bt-barber.com.br`). Se ainda não tem, pode usar a URL provisória do Vercel (`seu-projeto.vercel.app`) — atualize depois.
8. **MCC (categoria do comerciante):** escolha `7230 – Beauty/Barber Shops` . É o código mais alinhado.
9. **Ticket médio:** `R$ 50` . Volume mensal estimado: `R$ 1.000 – R$ 5.000` (começa baixo, depois pede aumento).
10. **Documento de identificação:** foto/scan do RG ou CNH + selfie segurando o documento. Stripe usa AI pra validar.
11. Ao final, "**Submit**".

A aprovação leva **de algumas horas a 2 dias úteis**. Você recebe um email quando aprovado. Enquanto isso, dá pra continuar usando Test mode pra desenvolver e testar. Os passos 6 a 13 deste guia podem ser feitos em paralelo com a espera.

⚠ **Atenção:** Se a Stripe pedir documentos extras (comprovante de endereço, contrato social), vai ter um link no e-mail. Sobe os arquivos pelo próprio dashboard, não responde por e-mail.

2 Habilitar PIX e cartão

Por padrão o cartão já vem ligado, mas **PIX precisa ser ativado manualmente**, e o processo é separado em Test mode e Live mode.

2.1 Ativar em Test mode (pra você testar local)

1. No dashboard, garanta que está em **TEST MODE** (toggle no canto superior direito).
2. Acessa https://dashboard.stripe.com/test/settings/payment_methods (ou: **Settings** → **Payments** → **Payment methods**).
3. Procura o card **Pix** na lista.

VOCÊ DEVE VER

Uma lista com cards de cada método: Card, Pix, Boleto, etc. Cada um tem um toggle ou botão "Turn on".

4. Clica em "**Turn on**" no card do Pix.
5. Stripe pode pedir uma confirmação rápida com o nome do produto. Em test mode é instantâneo.
6. Confirma também que **Card** está ligado (geralmente já vem).

2.2 Ativar em Live mode

Faz o mesmo processo em https://dashboard.stripe.com/settings/payment_methods (URL sem `/test/`).

⚠ **Atenção:** Em Live mode, ativar PIX pode levar de minutos a 24h porque a Stripe valida que sua conta está em compliance pra processar PIX no Brasil. Você recebe email quando estiver liberado.

3 Configurar branding e descritor de extrato

Esse passo determina como o cliente vê a BT-Barber: no email de recibo automático da Stripe, na fatura do cartão, e nas telas de checkout hospedadas (caso use no futuro).

3.1 Branding (logo + cores)

1. **Settings** → **Branding** (em qualquer modo — vale pros dois).
2. **Icon / Logo:** PNG de no mínimo 128x128, recomendado 512x512, fundo transparente. Use o ícone scissors do app (você pode exportar o SVG em `app/icon.svg` e converter pra PNG via cloudconvert.com).
3. **Brand color:** `#A05A2C` (marrom-cognac do tema light do app — bate com o gradiente da marca).
4. **Accent color:** `#1FB6FF` (azul-elétrico do tema dark — só aparece em alguns componentes secundários).

PREVIEW

A direita da tela mostra um preview do recibo de cobrança. Confere que o logo aparece centralizado e a cor primária está correta no cabeçalho.

3.2 Descritor de extrato

1. Ainda em **Settings** → **Public details** (ou **Public business info**, depende da versão).
2. **Statement descriptor:** `BT-BARBER` (9 chars, máx 22, só letras/números/espaco/hífen)
3. **Shortened descriptor / Prefix:** `BT-BARBER` (máx 10 chars)
4. **Customer support phone:** telefone real (Stripe exige pra ativação). Pode ser celular.
5. **Customer support email:** `contato@bt-barber.com.br`. Crie essa caixa no Google Workspace, Zoho ou no provedor do seu domínio antes de finalizar este passo.
6. **Save.**

O *statement descriptor* é o que aparece na fatura do cartão do cliente (ex.: "BT-BARBER"). No PIX, o que aparece é a razão social cadastrada no KYC — então capricha no nome legal lá.

4 Pegar as API keys de produção

Depois que a conta foi aprovada (passo 1) e PIX/cartão estão ligados em Live (passo 2), as Live keys ficam disponíveis.

1. No dashboard, garanta que está em **LIVE MODE**.
2. **Developers** → **API keys** (ou dashboard.stripe.com/apikeys).

VOCÊ DEVE VER

Duas chaves: **Publishable key** (`pk_live_...`) e **Secret key** (`sk_live_...`). Em test mode são `pk_test_ / sk_test_`.

3. Clica em **"Reveal live key"** no card da Secret key.
4. Copia ambas — guarda no seu gerenciador de senhas ou num arquivo de notas seguro.

⚠ **Crítico:** A Secret key dá acesso TOTAL à sua conta Stripe — quem tiver essa chave pode disparar pagamentos, refunds, ler todos os clientes. Nunca commita no Git, nunca cola em chat público, nunca posta em screenshot. Se vazar, gira no mesmo dashboard com "Roll key".

Anota:

- `STRIPE_SECRET_KEY = sk_live_...`
- `NEXT_PUBLIC_STRIPE_PUBLISHABLE_KEY = pk_live_...`

5 Adicionar conta bancária para receber repasses

A Stripe transfere automaticamente seu saldo pra essa conta. É dali que você puxa o dinheiro pra pagar as barbearias parceiras (PIX/TED manual).

1. Em Live mode, vai em **Settings** → **Payouts** → **Bank accounts and scheduling** (dashboard.stripe.com/settings/payouts).
2. Clica em **"Add bank account"**.
3. Preenche: banco, agência, número da conta, tipo (corrente/poupança). Stripe Brasil aceita PIX e TED.
4. **Schedule:** escolhe **Daily** (recomendado pra começar — fluxo de caixa rápido) ou **Weekly** (todas as quartas, por exemplo).
5. Salva. Stripe pode enviar um depósito de centavos pra confirmar a conta. Se acontecer, você confirma o valor exato pelo dashboard.

Em Live mode novo, o primeiro payout costuma ter um **delay de 7-14 dias** (Stripe segura o dinheiro como anti-fraude). Depois cai pra T+1 ou T+0 dependendo do volume.

6 Comprar/configurar domínio

Sem domínio, Stripe não aceita finalizar o KYC e Vercel só te dá uma URL `.vercel.app` (que dá pra usar no curtíssimo prazo, mas gera desconfiança no cliente).

6.1 Comprar (se ainda não tem)

- **.com.br**: <https://registro.br> (~R\$ 40/ano, exige CPF ou CNPJ).
- **.com**: [Cloudflare Registrar](#) (preço de custo, sem markup) ou [Namecheap](#).

6.2 Apontar pro Vercel (faz no passo 10)

Você só configura o DNS depois de criar o projeto na Vercel — quando Vercel te diz exatamente qual `CNAME` ou `A record` configurar. Por enquanto, só compra e guarda.

Sugestão de nome: `bt-barber.com.br`. Se já estiver ocupado, `btbarber.com.br` ou `btbarber.app`. Evita hífens no domínio raiz se possível — usuário escreve errado.

7 Subir código no GitHub

Vercel deploya direto do GitHub. Se o repo já está lá, pula pro passo 8.

1. Em github.com/new, cria um repo **privado** chamado `bt-barber`.
2. Não inicializa com README/LICENSE — você já tem o código.
3. No terminal local, dentro de `/Users/tecnologia/projetos/BT-Barber`:

```
git remote add origin git@github.com:<seu-usuario>/bt-barber.git
git branch -M main
git push -u origin main
```

⚠ **Atenção:** Antes de empurrar, confere que `.env` está no `.gitignore`: `cat .gitignore | grep .env`. Se não estiver, adiciona — **sua secret key da Stripe não pode ir pro GitHub**.

8 Importar projeto na Vercel

1. vercel.com → entra com a conta GitHub.
2. Topo da tela → "Add New" → "Project".
3. Lista os repos do GitHub. Se o repo `bt-barber` não aparece, clica em "Adjust GitHub App Permissions" e dá acesso.
4. Localiza o `bt-barber` → clica em "Import".

TELA DE CONFIGURAÇÃO

Vercel detecta Next.js automaticamente. **Não mexe** em Build & Output Settings — defaults estão corretos.

8.1 Antes de clicar em Deploy

Não clica em Deploy ainda — primeiro adiciona as variáveis de ambiente (passo 9), senão o build falha.

Expanda a seção "Environment Variables" abaixo de Build Settings.

9 Configurar variáveis de ambiente na Vercel

Adiciona **uma a uma** usando o formulário "Key / Value" e clicando em "Add" pra cada. Marque "All Environments" (Production, Preview, Development) — não tem porque limitar ainda.

9.1 Gere os secrets primeiro

No seu terminal local:

```
# Gera 2 secrets aleatórios — anota cada um
openssl rand -hex 32 # → vai ser NEXTAUTH_SECRET
openssl rand -hex 32 # → vai ser CRON_SECRET (outro, diferente)
```

9.2 Tabela de variáveis

Nome	Valor	Origem
DATABASE_URL	postgresql://postgres.<ref>:<senha>@aws-1-sa-east-1.pooler.supabase.com:6543/postgres?pgbouncer=true	Mesmo do <code>.env</code> local
DIRECT_URL	postgresql://postgres.<ref>:<senha>@aws-1-sa-east-1.pooler.supabase.com:5432/postgres	Mesmo do <code>.env</code> local

Nome	Valor	Origem
GOOGLE_CLIENT_ID	<numero>– <hash>.apps.googleusercontent.com	Mesmo do .env local
GOOGLE_CLIENT_SECRET	GOCSPX–...	Mesmo do .env local
NEXTAUTH_URL	https://<seu-dominio> (sem barra final)	Seu domínio do passo 6
NEXTAUTH_SECRET	O hex de 64 chars que você gerou	openssl rand -hex 32
STRIPE_SECRET_KEY	sk_live_...	Passo 4
NEXT_PUBLIC_STRIPE_PUBLISHABLE_KEY	pk_live_...	Passo 4
STRIPE_WEBHOOK_SECRET	whsec_... (você ainda não tem — coloca placeholder e atualiza no passo 11)	Passo 11
CRON_SECRET	O segundo hex de 64 chars que você gerou	openssl rand -hex 32
PLATFORM_ADMIN_EMAILS	patrick.almeida@sttart.com.br	Seu email Google (o que loga no app)

🚨 **Crítico:** Sem PLATFORM_ADMIN_EMAILS, NINGUÉM consegue marcar payouts para as barbearias. Esse é o ponto que decide quanto cada loja já recebeu. Confirma que adicionou.

⚠️ **Atenção:** NEXTAUTH_URL sem barra final. Domínios com / no final geram URLs //api/... que quebram OAuth de forma intermitente.

9.3 Deploy inicial

Depois de adicionar todas, clica em **"Deploy"**. Vercel builda o projeto. Deve passar em 2-4 minutos.

Se o build falhar com erro de Prisma *"Can't reach database server"*, é porque o Supabase pooler IPv4 está bloqueado. Confirma que DATABASE_URL usa aws-1-sa-east-1.pooler.supabase.com:6543 (não db.<ref>.supabase.co).

Conectar domínio na Vercel

1. No seu projeto na Vercel, vai em **Settings** → **Domains**.
2. No campo de input, digite o domínio (ex.: `bt-barber.com.br`) e clica em **"Add"**.
3. Adiciona também `www.bt-barber.com.br` (Vercel cria redirect automático).

O QUE VERCEL MOSTRA

Vercel exibe os registros DNS que você precisa configurar no provedor do domínio. Algo como:

- **Apex (sem www):** A record apontando pra `76.76.21.21`
- **www:** CNAME apontando pra `cname.vercel-dns.com`

10.1 Configurar DNS no Registro.br

1. Loga em registro.br → seu domínio.
2. **"Editar DNS"**.
3. Adiciona os registros que a Vercel passou. Salva.

Propagação DNS pode levar de minutos a 24h. Geralmente são minutos. Vercel atualiza o status quando reconhece. SSL (HTTPS) é emitido automaticamente em ~5 min depois disso.

Se usa Cloudflare Registrar, o DNS já é automático na Cloudflare. No painel deles → DNS → Add Record com os mesmos valores. Mas **desliga o proxy (nuvem laranja)** nos registros — Vercel não funciona com proxy do Cloudflare.

10.2 Atualizar NEXTAUTH_URL

Volta em **Vercel** → **Settings** → **Environment Variables** e confirma que `NEXTAUTH_URL` está como `https://bt-barber.com.br` (com o domínio real, sem barra final). Se estava com a URL provisória `.vercel.app`, edita e clica em **"Save"**. Vercel pede pra re-deployar — clica em **"Redeploy"**.

Criar webhook da Stripe (modo Live)

Sem webhook, status de pagamento nunca chega no app — pedidos pagos ficam pra sempre como PENDING. Faz isso depois que o domínio está respondendo (passo 10 finalizado).

1. Stripe dashboard em **LIVE MODE**.
2. **Developers** → **Webhooks** (ou dashboard.stripe.com/webhooks).
3. **"Add endpoint"**.

11.1 Preencher o formulário

Campo	Valor
Endpoint URL	<code>https://<seu-dominio>/api/webhooks/stripe</code>
Description	BT-Barber webhook (production)
Listen to	Events on your account
API version	Deixa o default (Stripe usa a sua versão da SDK)
Select events	Marca exatamente esses 4: <code>payment_intent.succeeded</code> <code>payment_intent.payment_failed</code> <code>payment_intent.canceled</code> <code>charge.refunded</code>

4. "Add endpoint".

11.2 Pegar o signing secret

1. Na tela do endpoint criado, role até a seção **"Signing secret"**.
2. Clica em **"Reveal"** (ou "Click to reveal").
3. Copia o valor — começa com `whsec_`.

11.3 Atualizar na Vercel

1. Vercel → seu projeto → **Settings** → **Environment Variables**.
2. Localiza `STRIPE_WEBHOOK_SECRET` (você criou como placeholder no passo 9).
3. Clica nos 3 pontinhos → **Edit** → cola o `whsec_...`.
4. **Save**.
5. Vercel pede redeploy → **Redeploy**.

Se você for ter ambiente *preview* separado depois, crie um endpoint diferente na Stripe pra cada (com URL e secret próprios). Por enquanto só prod basta.

12 Configurar redirect do Google OAuth

Login com Google só funciona em domínios cadastrados no Google Cloud Console. Se esquecer disso, você loga em prod e vê erro `"redirect_uri_mismatch"`.

1. console.cloud.google.com → seleciona seu projeto (o mesmo onde criou o OAuth client).
2. **APIs & Services** → **Credentials**.

3. Na lista **OAuth 2.0 Client IDs**, clica no client que você usa (provavelmente "Web client 1").

TELA DE EDIÇÃO

Mostra "**Authorized JavaScript origins**" e "**Authorized redirect URIs**" com os endereços de dev (`http://localhost:3000`).

12.1 Adicionar origens e redirects de produção

1. Em **Authorized JavaScript origins**, clica em "+ ADD URI" e adiciona: `https://<seu-dominio>`
2. Em **Authorized redirect URIs**, clica em "+ ADD URI" e adiciona: `https://<seu-dominio>/api/auth/callback/google`
3. **Save** (botão azul no rodapé).

Mantém os entries de `localhost` — você ainda vai usar pra desenvolver.

⚠ **Atenção:** Google às vezes leva 5-10 minutos pra propagar mudanças no OAuth. Se ao testar login você ver erro mismatch, espera um pouco e tenta de novo.

13 Preencher placeholders das páginas legais

As páginas `/termos`, `/privacidade` e `/cancelamento` têm marcadores [...] que você precisa trocar por dados reais antes de operar.

13.1 Lista de placeholders

Arquivo	Placeholder	Substituir por
app/termos/page.tsx	[Razão Social, CNPJ]	Ex.: <i>STTART Tecnologia LTDA, CNPJ 00.000.000/0001-00</i>
	[Endereço]	Endereço da sede
	[Cidade/UF]	Sua cidade — foro de eleição em caso de disputa
app/privacidade/page.tsx	[Razão Social, CNPJ] e [Endereço]	Mesmo dos termos
	[Nome] (do DPO)	Você mesmo, se MEI/sozinho. Caso contrário, alguém da equipe.

13.2 E-mails citados

Os textos referenciam `contato@bt-barber.com.br` e `dpo@bt-barber.com.br`. Você pode:

- **Criar essas caixas** (Google Workspace ~R\$ 30/mês, Zoho Mail tem plano grátis pra 5 usuários, Proton Mail também).
- **OU** trocar pelo email real que você quer usar (ex.: `patrick.almeida@sttart.com.br`).

13.3 Commit + redeploy

Depois de editar:

```
git add app/termos/page.tsx app/privacidade/page.tsx app/cancelamento/page.tsx
git commit -m "docs: preencher dados legais"
git push
```

Vercel detecta o push e redeploys automaticamente. Confere as 3 páginas no domínio de prod depois.

14 Verificar Vercel Cron

O arquivo `vercel.json` que está no repo configura um cron a cada 5 minutos batendo em `/api/cron/cleanup-bookings`. Sem isso, slots abandonados pelo cliente ficam ocupados pra sempre.

1. Vercel → seu projeto → **Settings** → **Cron Jobs**.

VOCÊ DEVE VER

Uma entrada listando `/api/cron/cleanup-bookings` com schedule `* / 5 * * * *` (every 5 minutes) e status **Active**.

2. Se não aparecer: verifica que `vercel.json` está na raiz do repo. Faz um redeploy via **Deployments** → **último deploy** → **"..."** → **Redeploy**.
3. Pra testar manualmente, clica nos 3 pontinhos do cron job → **"Trigger now"**. Aparece nos logs em **Logs** → **seu cron**. Resposta esperada: `{"deleted": 0}`.

Vercel injeta automaticamente o header `Authorization: Bearer $CRON_SECRET` em jobs agendados, então o handler aceita sem você precisar configurar nada extra.

Última verificação antes de divulgar. Faz uma transação real de baixíssimo valor pra exercitar todo o pipeline.

15.1 Cria um produto/serviço de teste

1. Loga no app de prod com seu Google.
2. Vai em `/admin/<sua-loja>/products` e cria um produto: nome "*Café teste*", preço **R\$ 1,00**, estoque 5, qualquer imagem.

15.2 Compra com cartão real

1. Acessa a tela do cliente: `/barbershop/<sua-loja>`.
2. Aba "**Loja**" → adiciona o "Café teste" ao carrinho → finaliza compra.
3. No checkout, escolhe **Cartão** e usa um cartão real seu (NÃO usa o cartão de teste 4242... em prod — só funciona em test mode).
4. Paga.

15.3 O que verificar

- ☐ Email com recibo da Stripe chegou no seu Gmail (Stripe envia automaticamente porque setamos `receipt_email`).
- ☐ Em `/orders`, o pedido aparece com badge **PAGO** e status **CONFIRMED**.
- ☐ Em `/admin/<sua-loja>/orders`, o pedido está listado e o owner pode mudar pra READY/COMPLETED.
- ☐ Em `/admin/<sua-loja>/payouts`, "A receber" subiu R\$ 0,95 (R\$ 1,00 menos 5% de fee da plataforma).
- ☐ Em **Vercel** → **Logs**, aparece a entrada do webhook (filtra por `scope:"stripe-webhook"` ou só vê o POST em `/api/webhooks/stripe` retornando 200).
- ☐ Em **Stripe Dashboard** → **Payments** em Live mode, o pagamento aparece com status "Succeeded".

15.4 Testa cancelamento + estorno

1. Volta em `/orders`, clica em "**Cancelar**".
- ☐ Toast diz "*Estorno de R\$ 0,95 processado*".
 - ☐ Em ~30s, status do Payment vira **REFUNDED** em `/admin/<sua-loja>/orders` (o webhook `charge.refunded` processou).
 - ☐ Stock do produto voltou pra 5 (vai em `/admin/<sua-loja>/products` e confere).
 - ☐ Em 2-7 dias o R\$ 0,95 volta no extrato do cartão.

Se algum item falhar, primeiro lugar pra investigar é **Vercel Logs** filtrando por `"level":"error"`. Os logs estão em formato JSON estruturado, então dá pra usar o filtro built-in da Vercel.

16 Hardening pós-launch (recomendado)

Não bloqueia o lançamento, mas é forte recomendação fazer na primeira semana.

16.1 Sentry (monitoramento de erros)

Sem Sentry, erro de webhook é descoberto quando cliente reclama. Com Sentry, você recebe email/Slack na hora.

1. sentry.io → cria conta (free tier dá 5k erros/mês).
2. Cria projeto Next.js. Sentry te dá um DSN (`https://<public>@<org>.ingest.sentry.io/<id>`).
3. No terminal local: `pnpm add @sentry/nextjs`.
4. `npx @sentry/wizard@latest -i nextjs` — segue o wizard.
5. No `app/_lib/log.ts`, troque `console.error` por `Sentry.captureException`. Os call sites continuam iguais.
6. Adiciona `SENTRY_DSN` nas envs da Vercel.
7. Commit + push + redeploy.

16.2 Backups do Supabase

Free tier dá só 7 dias de backup diário. Sem PITR (Point-In-Time Recovery), recuperar de uma corrupção é doloroso.

1. [Supabase](#) → [Settings](#) → [Billing](#).
2. Upgrade pro **Pro** (US\$ 25/mês). Habilita **PITR** de 7 dias.

16.3 Aumentar limite de transação

Stripe Brasil novas contas começam com limite de ~R\$ 5k/mês. Após 30 dias com transações limpas (sem chargebacks), peça aumento em **Settings** → **Limits** → **Request limit increase**.

16.4 Configurar alertas no Stripe

1. **Settings** → **Team** → **Notifications**.
2. Marca "**Disputed payments**" e "**Failed payouts**".
3. Define o email de alerta.

16.5 NFe (futuro)

Stripe Brasil **não emite nota fiscal**. Pra MVP dá pra operar sem, mas algum cliente vai pedir comprovante fiscal eventualmente. Quando isso acontecer, integre com [NFe.io](#) ou [eNotas](#) via webhook

(gera NFe de serviço a cada `charge.succeeded`).

Apêndice — Recovery em caso de problema

Webhook não chega

- Stripe Dashboard → **Developers** → **Webhooks** → **seu endpoint** → **aba "Events"**: mostra todas as tentativas. Se aparece status 4xx/5xx, clica no evento pra ver o body da resposta.
- Causas comuns: `STRIPE_WEBHOOK_SECRET` errado na Vercel (signature mismatch → 400), domínio não respondendo (timeout), handler levantando exception (500).
- Pra reprocessar manualmente: **Webhook** → **Events** → **click no evento** → **"Resend"**.

Pagamento ficou stuck em PENDING

- Confere se o webhook chegou (passo acima). Se não chegou, é o motivo.
- O app tem polling fallback de 3s — abre o pedido em **/orders**, espera ~10s, deve atualizar mesmo sem webhook.
- Manual: Stripe Dashboard → Payment → "Refund" se o cliente nunca foi atendido.

Saldo Stripe ≠ "A receber" no admin

- Saldo Stripe é o que sobrou após payouts pra sua conta bancária. "A receber" no app é o que você deve às barbearias.
- Reconciliação: **Stripe** → **Reports** → **Payments** exporta CSV. Compara com a tabela `Payment` via Prisma Studio.

Cliente perdeu dinheiro (pagou e não foi atendido)

- Owner cancela o pedido pelo app — refund automático sai.
- OU você manualmente: Stripe Dashboard → Payment → Refund (full ou partial).

Documento gerado por Claude Code para Patrick · BT-Barber v0.1.0 · Maio de 2026.